Stockd.

Software Requirements Specification (SRS)

CMSI 4071 — Senior Project I | Fall 2026

Mason Lui • Leila Nawas • Paige Inoue

8 October 2026 | Version 1.0

Project scope: automatic food detection, refrigerator inventory management, expiration tracking, and recipe suggestions. Pantry tracking is not part of the required baseline.

5.0 Requirements Specification

5.1 Introduction

Stockd is a smart refrigerator inventory system demonstrated using a small refrigerator enclosure. A Raspberry Pi 5 and wide-angle camera monitor food inside the unit. A magnetic door-contact sensor signals door-opening and door-closing events; while the door is open, the capture component collects images for food recognition. Recognition results are compared with existing inventory records to propose food additions or removals. The application permits confirmation and correction of uncertain detections, tracks expiration dates, displays alerts, and recommends recipes based on recorded ingredients. A networked backend and persistent database synchronize the inventory displayed on a phone or tablet. Figure 1 summarizes the principal components; Sections 5.2–5.5 describe the component breakdown, functional requirements, performance requirements, and operating environments.

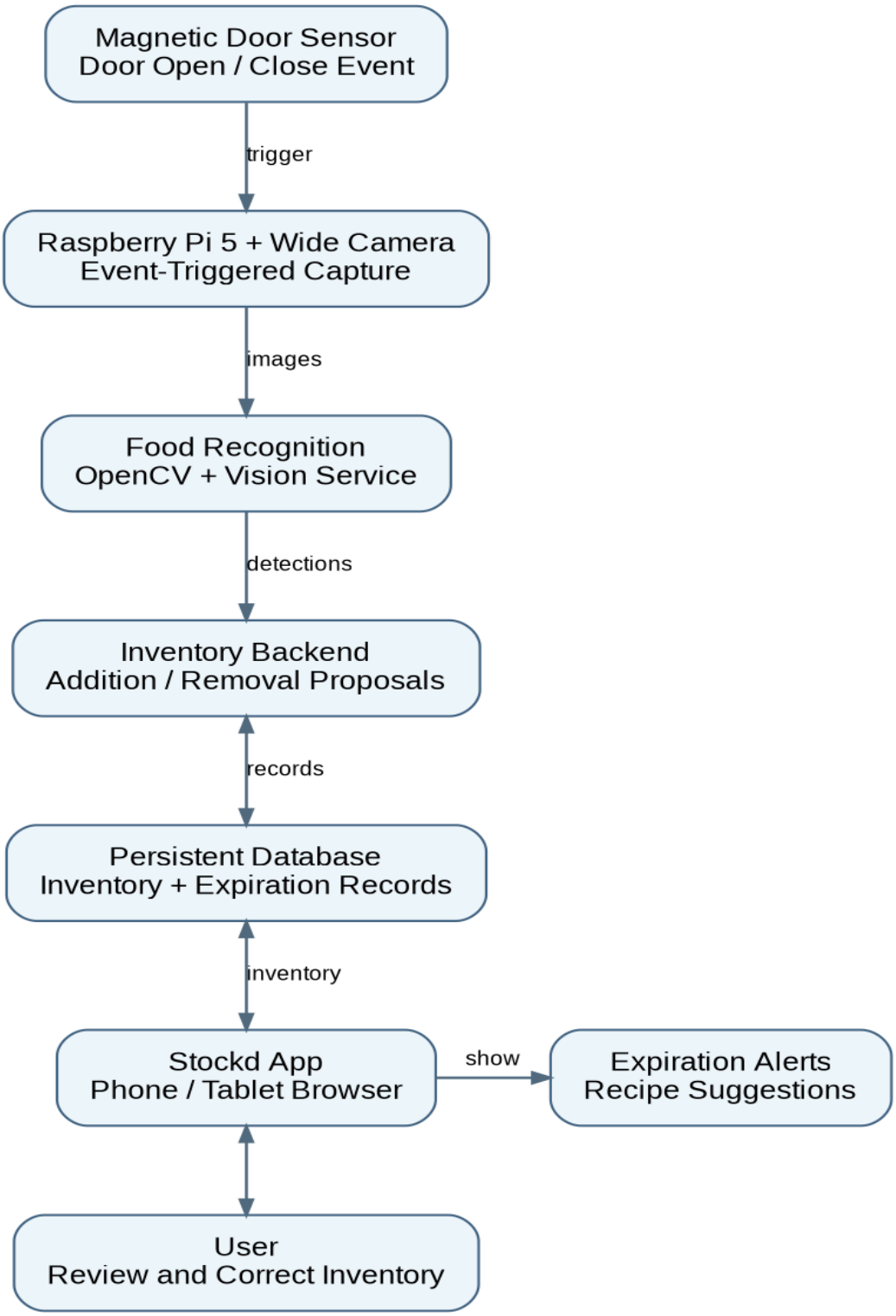


Figure 1. Stockd high-level architecture. A magnetic door sensor triggers image capture; food recognition and inventory services update the database and application.

5.2 CSCI Component Breakdown

CSCI Stockd comprises the following Computer Software Components (CSCs). Each CSC is divided into Computer Software Units (CSUs) for requirements and later design traceability.

5.2.1 Image Capture CSC

5.2.1.1 Camera Interface CSU

5.2.1.2 Automatic Capture Event CSU

5.2.2 Food Recognition CSC

5.2.2.1 Image Analysis CSU

5.2.2.2 Review and Correction CSU

5.2.3 Inventory Management CSC

5.2.3.1 Inventory Operations CSU

5.2.3.2 Automatic Addition/Removal Tracking CSU

5.2.4 Expiration Management CSC

5.2.4.1 Date Estimation CSU

5.2.4.2 Expiration Status CSU

5.2.5 Application Interface CSC

5.2.5.1 Inventory View CSU

5.2.5.2 Item Detail and Editing CSU

5.2.6 Data Management CSC

5.2.6.1 Persistence CSU

5.2.6.2 Data Validation CSU

5.2.7 Notification CSC

5.2.7.1 Alert Evaluation CSU

5.2.7.2 Alert Presentation CSU

5.2.8 Recipe Suggestions CSC

5.2.8.1 Recipe Suggestions CSU

5.2.8.2 Ingredient Matching CSU

5.3 Functional Requirements by CSC

Each numbered shall statement defines one testable required function. Automatic item detection and recipe suggestions are baseline capabilities; pantry tracking is excluded from baseline acceptance testing.

5.3.1 Image Capture

This group defines externally verifiable functions of the image capture component.

5.3.1.1 The capture subsystem shall activate image capture following a door-open event from the magnetic door-contact sensor.

5.3.1.2 The capture subsystem shall acquire a still image from the configured refrigerator camera.

5.3.1.3 The capture subsystem shall associate a unique identifier with each captured image.

5.3.1.4 The capture subsystem shall associate a capture timestamp with each captured image.

5.3.1.5 The capture subsystem shall reject an image that contains no readable image data.

5.3.1.6 The capture subsystem shall report camera-access failures to the application.

5.3.1.7 The capture subsystem shall show whether its camera connection is available.

5.3.1.8 The capture subsystem shall submit a captured image for recognition processing.

5.3.1.9 The capture subsystem shall report an automatic capture-in-progress state to the application.

5.3.1.10 The capture subsystem shall prevent duplicate submission of the same capture request.

5.3.1.11 The capture subsystem shall support a retry following a failed image capture.

5.3.1.12 The capture subsystem shall associate each captured image with the demonstration refrigerator.

5.3.1.13 The capture subsystem shall indicate when an automatic capture attempt fails.

5.3.1.14 The capture subsystem shall timestamp each automatically triggered capture event.

5.3.1.15 The capture subsystem shall recognize door-open signals from the magnetic door-contact sensor.

5.3.2 Food Recognition

This group defines externally verifiable functions of the food recognition component.

5.3.2.1 The recognition subsystem shall accept a captured image for analysis.

5.3.2.2 The recognition subsystem shall return an explicit processing status for each submitted image.

5.3.2.3 The recognition subsystem shall produce zero or more candidate food labels from a supported image.

5.3.2.4 The recognition subsystem shall associate a confidence value with each candidate label.

5.3.2.5 The recognition subsystem shall present detected food candidates for user review.

5.3.2.6 The recognition subsystem shall distinguish between a candidate identification and a user-confirmed identification.

5.3.2.7 The recognition subsystem shall allow a user to accept a proposed identification.

5.3.2.8 The recognition subsystem shall allow a user to reject a proposed identification.

5.3.2.9 The recognition subsystem shall allow a user to replace a proposed label with a manually entered label.

5.3.2.10 The recognition subsystem shall allow a user to add an item missed by image recognition.

5.3.2.11 The recognition subsystem shall signal when an image cannot be classified.

5.3.2.12 The recognition subsystem shall preserve user corrections when a review is saved.

5.3.2.13 The recognition subsystem shall flag low-confidence identifications for user review rather than silently applying an uncertain inventory change.

5.3.2.14 The recognition subsystem shall return detected food candidates for use in determining an inventory change.

5.3.3 Inventory Management

This group defines externally verifiable functions of the inventory management component.

5.3.3.1 The inventory subsystem shall create a record for a confirmed food item.

5.3.3.2 The inventory subsystem shall store a human-readable name for each inventory item.

5.3.3.3 The inventory subsystem shall store an item quantity.

5.3.3.4 The inventory subsystem shall allow a user to edit an item quantity.

5.3.3.5 The inventory subsystem shall allow a user to change an item name.

5.3.3.6 The inventory subsystem shall allow a user to manually add an inventory item.

5.3.3.7 The inventory subsystem shall allow a user to remove an inventory item.

5.3.3.8 The inventory subsystem shall display the active items assigned to a refrigerator.

5.3.3.9 The inventory subsystem shall distinguish active records from removed records.

5.3.3.10 The inventory subsystem shall record the date an item was added.

5.3.3.11 The inventory subsystem shall allow a user to mark an item as consumed.

5.3.3.12 The inventory subsystem shall allow a user to mark an item as discarded.

5.3.3.13 The inventory subsystem shall request confirmation before permanently deleting an inventory record.

5.3.3.14 The inventory subsystem shall prevent an item quantity from being set below zero.

5.3.3.15 The inventory subsystem shall preserve the latest confirmed quantity after an application restart.

5.3.3.16 The inventory subsystem shall allow item search by name.

5.3.3.17 The inventory subsystem shall allow items to be filtered by expiration status.

5.3.3.18 The inventory subsystem shall allow items to be sorted by expiration date.

5.3.3.19 The inventory subsystem shall distinguish newly added food items from items already present in the refrigerator inventory.

5.3.3.20 The inventory subsystem shall request user review when the direction of an inventory change cannot be determined reliably.

5.3.3.21 The inventory subsystem shall record a detected removal of a supported food item.

5.3.3.22 The inventory subsystem shall record a detected addition of a supported food item.

5.3.3.23 The inventory subsystem shall allow users to resolve possible duplicate inventory entries.

5.3.4 Expiration Management

This group defines externally verifiable functions of the expiration management component.

5.3.4.1 The expiration subsystem shall store an expiration or consume-by date when one is available.

5.3.4.2 The expiration subsystem shall allow users to enter a printed expiration date.

5.3.4.3 The expiration subsystem shall allow users to edit a stored expiration date.

5.3.4.4 The expiration subsystem shall allow users to indicate that an item has no known expiration date.

5.3.4.5 The expiration subsystem shall calculate the days remaining until a stored expiration date.

5.3.4.6 The expiration subsystem shall label items past their stored expiration date as expired.

5.3.4.7 The expiration subsystem shall label items within a configured alert window as expiring soon.

5.3.4.8 The expiration subsystem shall permit the user to select an item category for estimated consume-by dates.

5.3.4.9 The expiration subsystem shall mark estimated dates as estimates rather than printed dates.

5.3.4.10 The expiration subsystem shall allow the user to override an estimated consume-by date.

5.3.4.11 The expiration subsystem shall require manual date entry for leftovers when no estimate is available.

5.3.4.12 The expiration subsystem shall store the source of each expiration date (entered or estimated).

5.3.5 Application Interface

This group defines externally verifiable functions of the application interface component.

5.3.5.1 The application shall display a primary inventory screen.

5.3.5.2 The application shall display each item name in the inventory list.

5.3.5.3 The application shall display the quantity associated with each listed item.

5.3.5.4 The application shall display the expiration status of each item when known.

5.3.5.5 The application shall provide a control for adding an item.

5.3.5.6 The application shall provide a control for editing an item.

5.3.5.7 The application shall provide a control for removing an item.

5.3.5.8 The application shall display the status of automatically detected inventory changes.

5.3.5.9 The application shall provide an item detail view.

5.3.5.10 The application shall show validation errors next to affected input fields.

5.3.5.11 The application shall notify the user when a save action succeeds.

5.3.5.12 The application shall show an informative error when inventory cannot be loaded.

5.3.5.13 The application shall distinguish a loading state from an empty inventory.

5.3.5.14 The application shall allow navigation from the inventory screen to the alerts view.

5.3.5.15 The application shall present a confirmation interface for recognition review.

5.3.6 Data Management

This group defines externally verifiable functions of the data management component.

5.3.6.1 The data subsystem shall assign a unique identifier to each inventory record.

5.3.6.2 The data subsystem shall save confirmed inventory changes to persistent storage.

5.3.6.3 The data subsystem shall retrieve persisted records for inventory display.

5.3.6.4 The data subsystem shall record the last modification timestamp of an inventory item.

5.3.6.5 The data subsystem shall validate required fields before creating an inventory record.

5.3.6.6 The data subsystem shall reject records with invalid expiration-date formats.

5.3.6.7 The data subsystem shall report unsuccessful save operations to the calling component.

5.3.6.8 The data subsystem shall not treat a failed save as a successful update.

5.3.6.9 The data subsystem shall associate each inventory item with the demonstration refrigerator.

5.3.6.10 The data subsystem shall provide an authenticated connection for the application to retrieve inventory data.

5.3.6.11 The data subsystem shall permit inventory updates only through authenticated application requests.

5.3.6.12 The data subsystem shall keep item identity and expiration-date data linked to the same record.

5.3.7 Notification

This group defines externally verifiable functions of the notification component.

5.3.7.1 The notification subsystem shall evaluate active inventory items for expiration alerts.

5.3.7.2 The notification subsystem shall create an alert when an item enters the expiring-soon window.

5.3.7.3 The notification subsystem shall create an alert when an item passes its stored expiration date.

5.3.7.4 The notification subsystem shall include the item name in each expiration alert.

5.3.7.5 The notification subsystem shall include the item expiration date in each expiration alert when known.

5.3.7.6 The notification subsystem shall display current alerts in the application.

5.3.7.7 The notification subsystem shall distinguish unread alerts from read alerts.

5.3.7.8 The notification subsystem shall generate no more than one alert for the same item and expiration status during a calendar day.

5.3.7.9 The notification subsystem shall allow the user to dismiss an in-app alert.

5.3.7.10 The notification subsystem shall exclude removed items from new expiration alerts.

5.3.8 Recipe Suggestions

This group defines the required recipe recommendation functionality based on the recorded refrigerator inventory.

5.3.8.1 The recipe subsystem shall allow a user to request recipe suggestions.

5.3.8.2 The recipe subsystem shall identify active inventory items eligible for recipe suggestions.

5.3.8.3 The recipe subsystem shall display suggested recipe titles.

5.3.8.4 The recipe subsystem shall display ingredients required for a selected recipe.

5.3.8.5 The recipe subsystem shall indicate which recipe ingredients are missing from inventory.

5.3.8.6 The recipe subsystem shall identify recipes whose required ingredients are all recorded as available in the refrigerator inventory.

5.3.8.7 The recipe subsystem shall update recipe suggestions after a confirmed inventory change.

5.3.8.8 The recipe subsystem shall display a message when no matching recipes are available.

5.4 Performance Requirements by CSC

The numerical limits below are baseline acceptance targets for the demonstration prototype, measured using the test configuration specified in Section 5.5. The team should revise these through formal change control if prototype testing shows they are infeasible.

5.4.1 Image Processing

The recognition subsystem shall return a result or explicit error within 10 seconds for at least 95% of valid single-image requests under the agreed test network.

5.4.2 Inventory Load

The application shall display an inventory of 100 items within 3 seconds for at least 95% of requests under the agreed test network.

5.4.3 Save Response

The application shall acknowledge successful inventory edits within 3 seconds for at least 95% of valid edit requests under the agreed test network.

5.4.4 Capacity

The data subsystem shall support at least 100 active items per refrigerator without loss of required fields.

5.4.5 Persistence

The data subsystem shall preserve successfully committed records after an application restart.

5.4.6 Date Evaluation

The expiration subsystem shall assign an expiration status to 100 inventory records within 2 seconds on the designated test configuration.

5.4.7 Error Feedback

The application shall display a failed-request message within 3 seconds after receiving a backend error response.

5.4.8 Automatic Camera Capture

The capture subsystem shall return an image or camera error within 5 seconds after a supported automatic capture trigger on the designated hardware.

5.4.9 Notification Evaluation

The notification subsystem shall complete an expiration check of 100 items within 10 seconds on the designated backend.

5.4.10 Data Integrity

The data subsystem shall reject 100% of test cases with negative item quantities.

5.5 Project Environment Requirements

The following requirements define the expected demonstration configuration. Exact software versions and hosting vendors may be selected during implementation without changing the system's externally observable requirements.

5.5.1 Development Environment Requirements

- Source control shall use Git with the team GitHub repository.

- Development machines shall support the selected application framework, Python tooling, and backend development.
- The application shall be developed for a responsive browser interface suitable for phone and tablet displays.
- The backend shall provide an HTTP API for inventory, food recognition results, alerts, and recipe suggestions.
- The development environment shall include OpenCV and a selected food-recognition model or hosted vision service.
- The development environment shall include a relational database service; Supabase/PostgreSQL is the intended candidate.
- The team shall maintain access to the Raspberry Pi, camera, door-contact sensor, representative food items, and test images.

5.5.2 Execution Environment Requirements

- The execution hardware shall include a Raspberry Pi 5 (4GB), a compatible wide-angle camera, and a wired magnetic door-contact sensor.
- The execution environment shall provide Wi-Fi connectivity between the Raspberry Pi and backend service.
- Food recognition shall run through the selected local or hosted vision service accessible from the backend.
- The execution environment shall include a persistent database accessible to the backend.
- The client shall run in a current mobile or tablet browser with network access.
- The Raspberry Pi and its mains power supply shall remain outside the chilled refrigerator enclosure; the camera shall be mounted securely and protected against moisture.
- The required notification channel shall be in-app alerts. Operating-system push notifications are not required.

Baseline scope clarification

Required: event-triggered image capture, automatic candidate food detection, proposed addition/removal identification, manual review and correction, inventory persistence, expiration status and in-app alerts, and recipe suggestions. Food recognition accuracy and change-direction identification depend on available camera views; the system shall allow correction when items are obscured or uncertain.

Not required for baseline acceptance: pantry inventory, support for multiple refrigerators, push notifications, and identifying individual users. These capabilities may be considered future enhancements.